Interview Report — Containerized Fraud Detection API

Position Level: Junior / Trainee
Stack: Python · FastAPI · Docker · GitHub Actions · Scikit-learn

---

EASY — Conceptual & Surface-Level Understanding

Q1. What is the purpose of this project in one sentence?

Typical Answer:
It wraps a trained machine learning model inside a web API so that other systems can send transaction data and get back a fraud prediction in real time.

Q2. What is FastAPI and why did you choose it over Flask?

Typical Answer:
FastAPI is a modern Python web framework for building APIs. I chose it because it's faster than Flask, automatically generates documentation via Swagger UI, and has built-in data validation using Pydantic — which is useful for defining request/response schemas clearly.

Q3. What does "real-time inference" mean in this context?

Typical Answer:
It means the model makes a prediction immediately when a request arrives, rather than processing data in batches overnight. A client sends a JSON payload with transaction features, and the API responds with a fraud/not-fraud prediction within milliseconds.

Q4. What is Docker and what problem does it solve here?

Typical Answer:
Docker is a containerization tool that packages the application and all its dependencies into a single portable unit called a container. It solves the "works on my machine" problem — ensuring the API runs the same way in development, testing, and production.

Q5. What is a Dockerfile?

Typical Answer:
A Dockerfile is a text file with step-by-step instructions for building a Docker image. It defines the base OS image, copies the project files, installs dependencies, and sets the command to run the application.

Q6. What is Scikit-learn used for in this project?

Typical Answer:
Scikit-learn is used to train the fraud classification model. It provides ready-made algorithms like Logistic Regression, Random Forest, etc., as well as preprocessing tools. The trained model is saved and later loaded inside the FastAPI service to make predictions.

Q7. What is GitHub Actions?

Typical Answer:
GitHub Actions is a CI/CD platform built into GitHub. It lets you define automated workflows — for example, running your test suite every time you push code — using YAML configuration files stored in the `.github/workflows/` directory.

Q8. What does CI/CD stand for and what does it mean?

Typical Answer:
CI stands for Continuous Integration — automatically testing code when changes are pushed. CD stands for Continuous Deployment/Delivery — automatically deploying or preparing a release after tests pass. Together they reduce manual effort and catch bugs early.

Q9. What format does the API use to send and receive data?

Typical Answer:
JSON (JavaScript Object Notation). The client sends a POST request with a JSON body containing transaction features, and the API responds with a JSON object containing the prediction and optionally a confidence score.

Q10. What is a classification model?

Typical Answer:
A classification model predicts which category an input belongs to. In fraud detection, it predicts one of two classes: fraud (1) or not fraud (0), based on features like transaction amount, location, time, etc.

MEDIUM — Implementation & Design Thinking

Q11. How do you load the trained model inside FastAPI without reloading it on every request?

Typical Answer:
By loading the model once at startup using FastAPI's `lifespan` context manager or a startup event, then storing it in the app's state or a global variable. This avoids the heavy I/O of loading the model on every incoming request, which would be very slow.

```
# Example
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    app.state.model = joblib.load("model.pkl")
    yield

app = FastAPI(lifespan=lifespan)
```

Q12. How do you define and validate the request body in FastAPI?

Typical Answer:
Using a Pydantic `BaseModel`. You define a class with typed fields, and FastAPI automatically validates incoming JSON against it, returning a 422 error if required fields are missing or have wrong types.

```
from pydantic import BaseModel

class Transaction(BaseModel):
    amount: float
    merchant_category: str
    hour_of_day: int
    distance_from_home: float
```

Q13. What HTTP method do you use for the prediction endpoint and why?

Typical Answer:
POST, because we're sending data in the request body (transaction features) and triggering a server-side action (running inference). GET is not appropriate here since the features shouldn't be passed as URL query parameters — they can be large and sensitive.

Q14. What does your Docker image's base image look like and why does that choice matter?

Typical Answer:
I'd use something like `python:3.11-slim` as a base image. The `slim` variant is smaller than the full image, reducing build time and attack surface. Choosing the right base image balances size, security, and compatibility.

Q15. How do you handle model versioning — what happens when you retrain the model?

Typical Answer:
A simple approach is to save the model with a versioned filename (e.g., `model_v2.pkl`) or include the version in an environment variable that the app reads at startup. A more advanced approach uses a model registry. For this project, the model file is baked into the Docker image or mounted as a volume.

Q16. Walk me through what happens when a fraud prediction request hits your API end-to-end.

Typical Answer:
1. Client sends a POST request to `/predict` with JSON transaction data.
2. FastAPI validates the request body with Pydantic.
3. The validated data is converted into a NumPy array or DataFrame.
4. The pre-loaded Scikit-learn model runs `.predict()` or `.predict_proba()`.
5. The result is returned as a JSON response with the prediction label and optionally a probability score.

Q17. What does your GitHub Actions workflow actually do step by step?

Typical Answer:
The workflow triggers on every push to the main branch. It checks out the code, sets up a Python environment, installs dependencies from `requirements.txt`, and runs `pytest` to execute the test suite. If any test fails, the workflow fails and GitHub marks the commit as broken.

Q18. What kind of tests would you write for this project?

Typical Answer:
- **Unit tests:** Test preprocessing functions in isolation.
- **Integration tests:** Use FastAPI's `TestClient` to send mock requests to the `/predict` endpoint and assert the response format and status code.
- **Model tests:** Assert that the loaded model returns predictions in the expected format.

Q19. How do you pass configuration (like model path or port) to your Docker container without hardcoding it?

Typical Answer:
Using environment variables. In Docker, you pass them with `-e` flag or in a `docker-compose.yml`. Inside Python, you read them with `os.getenv()` or a library like `python-dotenv` for local development.

Q20. What is the difference between a Docker image and a Docker container?

Typical Answer:
An image is a read-only blueprint — like a class in OOP. A container is a running instance of that image — like an object. You can run multiple containers from the same image simultaneously.

Q21. What metrics would you use to evaluate your fraud detection model and why not just accuracy?

Typical Answer:
Accuracy is misleading for fraud detection because the dataset is heavily imbalanced — e.g., 99% of transactions are legitimate. A model that always predicts "not fraud" gets 99% accuracy but catches zero fraud. Better metrics are:

- **Precision:** Of predicted frauds, how many were real?
- **Recall:** Of actual frauds, how many did we catch?
- **F1-Score:** Harmonic mean of precision and recall.
- **AUC-ROC:** Overall model discrimination ability.

Q22. How do you expose the FastAPI container to external traffic?

Typical Answer:
By mapping the container's internal port to a host port using the `-p` flag in Docker: `docker run -p 8000:8000 my-api`. Inside the container, Uvicorn listens on port 8000, and this maps it to port 8000 on the host machine.

HARD — Architecture, Trade-offs & Depth

Q23. What are the risks of serializing your model with `joblib` or `pickle`, and how would you mitigate them?

Typical Answer:
Pickle/joblib files can execute arbitrary code when loaded, making them a security risk if the file comes from an untrusted source. Also, they may not be compatible across different Python or Scikit-learn versions. Mitigations include: using ONNX format for cross-compatibility, only loading models from trusted internal sources, and pinning exact library versions in `requirements.txt`.

Q24. How would you handle feature drift — the model was trained on old data but live transactions look different now?

Typical Answer:
I'd implement monitoring on the distribution of incoming features (e.g., using statistical tests like KS-test) and track prediction confidence over time. A significant shift would trigger an alert for model retraining. This is a data drift detection problem and tools like Evidently AI or WhyLogs can help automate it.

Q25. Your API is under heavy load — 10,000 requests per second. What would you change about this architecture?

Typical Answer:
- Add a **load balancer** and run multiple container replicas (e.g., via Kubernetes

or Docker Swarm).
- Use **async endpoints** in FastAPI if I/O-bound operations exist.
- Add a **message queue** (e.g., Kafka or Redis) for request buffering.
- Optimize the model with **ONNX Runtime** for faster inference.
- Add **caching** for repeated identical requests.
- Profile with tools like `locust` to identify the bottleneck first.

Q26. Why is fraud detection a hard machine learning problem specifically?

Typical Answer:
Several reasons: extreme **class imbalance** (fraud is rare), **adversarial behavior** (fraudsters adapt to evade detection), **concept drift** (fraud patterns change over time), **high cost of errors** (false negatives miss fraud; false positives annoy legitimate customers), and **latency requirements** (decisions must be made in milliseconds at payment time).

Q27. If you were to move this from a single Docker container to a production-grade system, what would you add?

Typical Answer:
- **Orchestration:** Kubernetes for scaling and self-healing.
- **Observability:** Logging (structured JSON logs), metrics (Prometheus + Grafana), and tracing (OpenTelemetry).
- **Model registry:** MLflow to track experiments and manage model versions.
- **API Gateway:** For authentication, rate limiting, and routing.
- **Database:** To log predictions for auditing and retraining pipelines.
- **Secrets management:** HashiCorp Vault or cloud provider secrets manager instead of env vars.

Q28. What is the difference between `CMD` and `ENTRYPOINT` in a Dockerfile?

Typical Answer:
Both define what runs when the container starts. `ENTRYPOINT` defines the fixed executable that always runs (e.g., `python`). `CMD` provides default arguments that can be overridden at runtime. When used together, `ENTRYPOINT` is the command and `CMD` is its default arguments. For a FastAPI app, a common pattern is:

```
ENTRYPOINT ["uvicorn"]
CMD ["main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Q29. How would you secure this API in a real deployment?

Typical Answer:
- **Authentication:** API keys or OAuth2/JWT tokens.
- **Input validation:** Already handled by Pydantic, but add business-logic validation too.
- **Rate limiting:** Prevent abuse via an API gateway or FastAPI middleware.
- **HTTPS:** TLS termination at the load balancer level.
- **Minimal Docker image:** Use distroless or slim images to reduce attack surface.
- **Dependency scanning:** Use tools like `pip-audit` or Snyk in the CI pipeline.

Q30. Explain the trade-off between model complexity and inference latency in a fraud detection API.

Typical Answer:
More complex models (e.g., deep neural networks, large ensembles) tend to be more accurate but slower to run. In real-time fraud detection, you often have a hard SLA of under 100ms. This means there's a trade-off: you may accept slightly lower accuracy in exchange for a simpler, faster model (e.g., Logistic Regression or a shallow tree). Alternatively, you can optimize a complex model using quantization, pruning, or ONNX export. The right choice depends on the business's tolerance for latency vs. false negatives.

Summary

Difficulty	Questions	Focus Area
Easy	Q1–Q10	Concepts, definitions, project overview
Medium	Q11–Q22	Implementation, API design, testing, Docker
Hard	Q23–Q30	Architecture, trade-offs, production thinking

Total: 30 questions across all angles of the project.